

A Quantum/Classical Example Oracle Separation for Making Things Up

Kenny Chen*

August 2026

Abstract

We study the power of quantum examples, as compared to classical examples, in the PAC learning framework. Here, we have two learning algorithms, both with access to quantum computation, but one gets quantum examples, whereas the other gets classical examples. It was previously unknown whether there were learning tasks that can be efficiently performed but not by the latter. Our primary result is to show that relative to an oracle, there are distributions that can be efficiently generated by a quantum learner with access to quantum examples, but not by a quantum learner with access to only classical examples, making progress to answering this question in the affirmative.

1 Introduction

In 1984, Valiant introduced the celebrated concept of Probably Approximately Correct (PAC) Learnability [Val84], which then sparked the fields of computational learning theory, machine learning theory, and influenced many others. A major question in this field then, was to determine what functions, or classes of functions were efficiently PAC learnable. A decade later, Shor presented his seemingly unrelated yet equally celebrated quantum algorithm for factoring integers [Sho97]. This spurred yet another field of questions, centered around the power of quantum computation. Namely, how much more powerful were quantum computers than classical computers, and what other problems admitted efficient quantum algorithms, when no efficient classical algorithm were known, akin to the problem of factoring. Soon, these two fields intertwined, and the question of whether quantum algorithms could be used to produce efficient PAC learners was raised. This question has been the subject of much study since. For example, Atici and Servedio [AS07] showed that using quantum examples and computations, one could learn and test juntas with quadratic advantage.

Having established that quantum computing could be used to create more efficient learning algorithms, there was also the question of how strong the separation could be. That is, were there classes of functions that were PAC-learnable quantumly efficiently, but not classically? One of the earlier works to touch on this was by Bshouty and Jackson [BJ99], who gave an efficient quantum algorithm for PAC learning DNF formulae over the uniform distribution: in contrast, there (currently) is no known classical algorithm for efficiently learning these formulae. However, there is also no impossibility result stating that such a classical efficient algorithm cannot exist.

*The University of Sydney, Email: kche5493@uni.sydney.edu.au

Given the difficulty of showing time separations, as a stepping stone the field turned its focus to query complexity. At a high level, this task involved deducing something about an unknown function, with algorithms equipped with black box query access to it. There were many results showing strong query separations between quantum and classical algorithms,¹ and so the related question was also raised in learning theory: specifically, *were there concept classes that were efficiently PAC learnable with few quantum samples, but were not efficiently PAC learnable with few classical samples?* This was the natural extension to the learning setting, where instead of being able to query a black box function, one would instead receive samples from an unknown function or distribution, with the task of learning these objects, or some properties of these objects. This question was eventually resolved in the negative by Arunachalam and De Wolf, who showed that for PAC learning under arbitrary distributions, quantum learners could only admit at most a constant factor in sample complexity advantage [AW18]. These results were later revisited by Salmon *et al.* [SSG24], who showed that if an algorithm had access to not just quantum samples, but also the underlying circuit that generated the quantum samples, then one could actually get a quadratic factor sample advantage in PAC learning under arbitrary distributions. However, this again only demonstrated a quadratic sample advantage, and not a separation result.

To the best of our knowledge, the first separation result came from Sweke *et al.*, who showed that quantum *computation* was strictly more powerful than classical computation in the learning setting [SSHE21]. Specifically, they exhibited a distribution class which, with classical examples, could be efficiently generated (see Definition 2.3) by an algorithm with quantum computation, but not by an algorithm with classical computation. They left open the question of whether quantum *examples* were strictly more powerful than classical examples (see Fig. 1). That is, given two quantum algorithms, one with quantum examples, and one with classical examples, did there exist functions or distributions that could be efficiently learned or generated by the former, but not by the latter. Thus, this raises the following question, the main topic of this paper:

How much more powerful are quantum examples, compared to classical examples, for learning functions with a quantum computer?

Though their focus was on proving computation separations, Sweke *et al.* did make several comments and directions for how a separation between quantum and classical examples might be proven. We discuss this in detail in Section 3.1, but provide a high-level overview here. One key result they showed was the following:

Theorem 1.1 ([SSHE21] Informal, see Theorem 3.1). *If a function class is efficiently PAC learnable, then the induced distribution class of tuples $(x, f(x))$ can be efficiently generated.*

To briefly explain the above, PAC learning requires an algorithm, after obtaining samples from an unknown function, to output a function that “approximates” the unknown function (for some appropriate notion of approximating). On the other hand, in the generating setting, one only requires that, given samples from an unknown distribution, an algorithm can efficiently output samples from a distribution that is “close” to the unknown distribution. Whilst one way of doing this in the above setting, where the underlying distribution is induced by a function, is to learn a function, this is not the only available method in the generating setting. They left the reverse direction open as a conjecture:

¹This is the field of quantum query complexity.

Conjecture 1.1 ([SSHE21], Informal, see Conjecture 3.1). *If a function class is not efficiently PAC learnable, then the induced distribution class cannot be efficiently generated.*

If proven true, this would potentially allow one to lift hardness results about a function class directly into a quantum/classical computation separation, provided the function class itself was efficiently quantumly learnable, but not efficiently classically learnable (both with classical examples). Sweke *et al.* speculated then, that should this be doable, it could be possible to use similar arguments to adjust the above two results to also show a separation between quantum and classical examples.

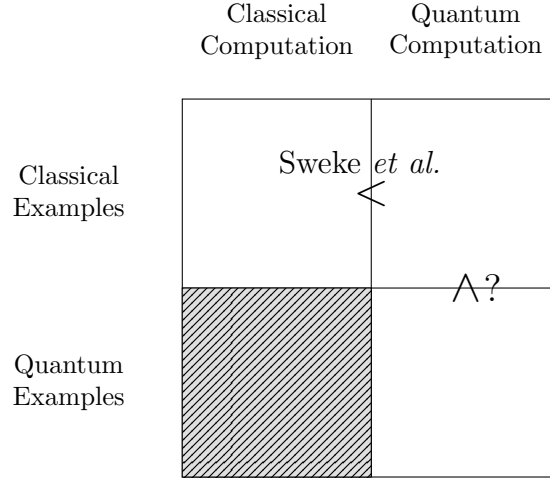


Figure 1: Sweke *et al.* showed a separation between learners with classical computation and learners with quantum computation when both had only classical examples. It is an open question as to whether there is a separation between quantum learners when one only has classical examples and the other has quantum examples.

1.1 Our Results

Our first result makes progress towards resolving Conjecture 1.1 in the negative. Namely, we show that under standard and widely accepted cryptographic assumptions, there is an oracle relative to which some function class that cannot be efficiently learned, yet the resulting distribution class can be efficiently generated.

Theorem 1.2 (Informal, see Theorem 3.2). *Relative to an oracle encoding a one-way permutation f , there exists a concept class of functions $\mathcal{C}_f$, which cannot be efficiently learned. However, for any $g \in \mathcal{C}_f$, the distribution class induced by these functions $(x, g(x))$, can be efficiently generated.*

This strongly suggests that new ideas are required in order to establish a separation for quantum and classical examples. Our second main contribution aims to resolve this question. Namely, we show that given access to an auxiliary function encoded via a classical oracle, there exists a distribution class that can be efficiently PAC generated by a quantum learner with quantum examples, but not by a quantum learner with only classical examples.

Theorem 1.3 (Informal, see Theorem 4.2). *Let $g: \{0, 1\}^n \rightarrow \{0, 1\}^n$ be a random boolean function. Then, there exists a distribution class $\mathcal{C}_g$ that can be efficiently generated by a quantum learner with quantum examples, when given oracle access to g . However, no quantum learner with only classical examples (and with oracle access to g) can efficiently generate this distribution class.*

Thus, we make significant progress on our guiding main question: to the best of our knowledge, our work is the first to show that in the context of learning (in the oracle setting), quantum examples allow efficient PAC learning of concept classes not efficiently learnable from classical examples, and thus are more powerful.

On the access to an auxiliary oracle. Both our main results rely on auxiliary oracle access to another function g . In other words, we have an unknown but queryable function g , which then induces a concept class $\mathcal{C}_g$. Then the learning algorithms can utilize both examples from the concept classes as well as queries to the function g in order to PAC learn/generate their concept classes. The resulting generators can also use oracle access to g to generate their outputs. We illustrate this interaction in Fig. 2. While endowing the learner and hypothesis with such an oracle access is not a standard assumption in PAC learning, we note that this has become a standard assumption in the field of quantum query complexity (and also in general complexity theory): indeed, proving oracle-less separations is one of the questions at the forefront of quantum complexity theory, and we currently lack the techniques to generally do so. As such, one standard assumption is to try and prove separations relative to an oracle. In fact, many separations between quantum and classical algorithms are oracle separations of this type, such as most boolean query separations, and the oracle separation between **BQP** and **PH** [RT22]. Hence, since our goal is to effectively show a separation between quantum and classical samples for the purpose of learning, we believe it is natural to show this separation in the context of an oracle separation.

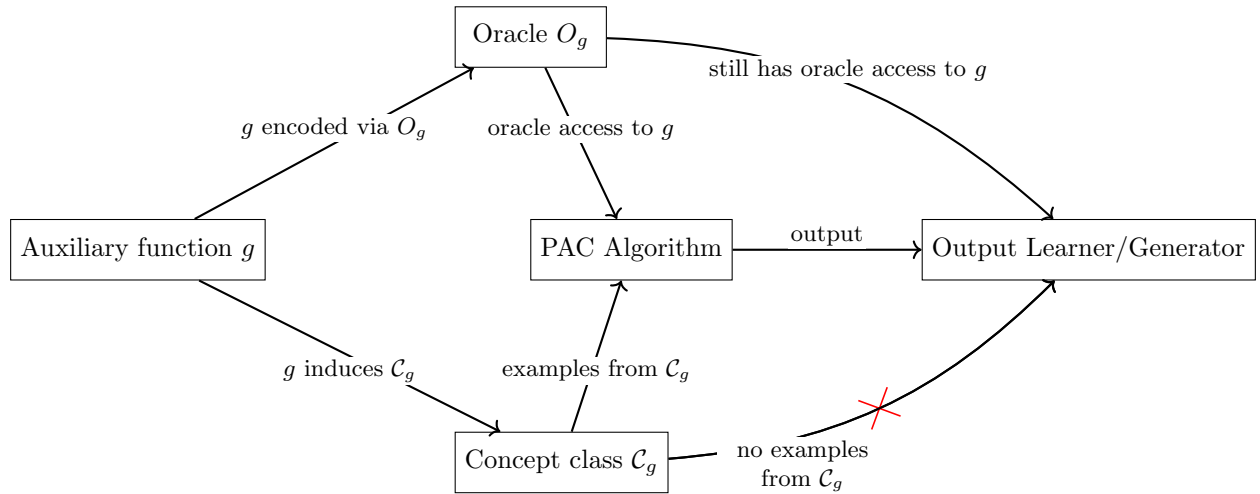


Figure 2: Illustration of our PAC learning setting. An auxiliary function g induces a concept class $\mathcal{C}_g$. The PAC learning algorithm has oracle access to g and examples from $\mathcal{C}_g$. They must output a learner/generator, which operates with oracle access to g , but no further examples from $\mathcal{C}_g$.

1.2 Our Techniques

We here provide a high-level overview of the techniques we use to obtain Theorem 1.2. To construct our concept class, we make use of *one-way permutations*, which informally, are permutations that are easy to evaluate, but (computationally) hard to invert. Given a one-way permutation g , and a randomly sampled nonzero string r , one can define *the hardcore predicate*, $B_r(x) = \langle g^{-1}(x), r \rangle \bmod 2$, which, roughly speaking, is a one-bit function “encapsulating” the hardness of g (i.e., hard to compute, even given g). Our function class then effectively consists of all of the hardcore predicates of this one-way permutation, and the distribution class is the distribution naturally induced by these predicates. It is well-known that evaluating the hardcore predicate given examples of the form $(g(x), r)$ is equivalent to being able to invert g itself. We then equip both learners with oracle access to the one-way permutation g . One learner is tasked with PAC learning the concept class, whilst the other is tasked with PAC generating this concept class. We then argue that the former task cannot be done efficiently; this task indeed implies that, for *most* strings x sampled uniformly at random, the learner should be able to correctly evaluate $B_r(x)$: we show that any learner that can do this efficiently can be used to efficiently invert the one-way permutation g , contradicting well-established hardness assumptions. In contrast, we show that the latter PAC generation task, with oracle access to g , can be done efficiently. The crucial difference is that in the former task, for a string x sampled uniformly at random, the learner effectively needs to compute $g^{-1}(x)$ for most x ’s, which (intuitively) requires inverting g . However in the latter task, the algorithm only needs to generate samples from a distribution close enough to the true induced distribution, and gets to “choose” the x ’s. That is, the key idea is to flip the table: the learner can uniformly sample a string s , and query the oracle to obtain $t := g(s)$. Now, we have $s = g^{-1}(t)$, so the learner looks like it “inverted” the function on this specific point t , and output a sample corresponding to the string t , instead of s . Because g is a one-way permutation, the distribution of $t = g(s)$ (for s uniform) is still uniform. (Importantly, the generator still does not know how to invert g , and does not have to.) The final remark is that since quantum-resilient one-way permutations are widely believed to exist, the hardness result extends to both quantum and classical learners.

Our second main contribution (Theorem 1.3) is a learning separation between quantum and classical examples relative to an oracle. The starting point is that, as shown by Kaplan *et al.* [KLLN16], Simon’s algorithm still succeeds with high probability on functions with a low number of pseudoperiods. We then first sample a random function $g: \{0, 1\}^n \rightarrow \{0, 1\}^n$. From this, any nonzero string a yields a periodic function f_a , defined by $f_a(x) = g(x) + g(x + a)$. This leads us to defining the concept class $\mathcal{C}_g$ induced by the random function g , which consists of all $2^n - 1$ such f_a ’s. Again, we give both learners oracle access to the random function g : in this case, both learners are quantum learners, the first with only classical examples of $f \in \mathcal{C}_g$, and the second with quantum examples. Both are then tasked with generating the distribution class induced by $\mathcal{C}_g$. We are able to show that, with high probability over the uniformly random function g , the function f has a low number of pseudoperiods for all real periods a . By the probabilistic method, this then implies there exists a function g such that all possible offsets a have a low enough pseudoperiod factor that Simon’s algorithm succeeds with high probability in recovering the period a . This implies that the learner with quantum examples can easily generate this distribution class, by recovering the period a , then uniformly sampling a string x , and querying the oracle g twice to obtain $g(x)$ and $g(x + a)$. But, by standard hardness results, recovering the period a from classical examples only requires an exponential number of examples, so this particular route is unavailable to the second learner.

We then show that any algorithm that does not recover the period will generate a distribution at total variation distance $1 - o(1)$ from the true distribution, implying that any quantum learner with only classical samples cannot efficiently generate this distribution class. This in turn, shows a separation between classical and quantum examples.

1.3 Related Work

As mentioned prior, there has been significant research on quantifying how much more powerful a quantum query is compared to a classical query in the field of query complexity. We refer to [Ham25] as a survey of this field. In particular, a result of Simon [Sim97] showing that quantum queries are strictly more powerful than classical queries in being able to detect a hidden period serves as a baseline for showing our PAC learning separation. Whilst we aim to show a separation between quantum and classical queries in the learning setting, such separations have also been studied in other contexts, such as graph algorithms [DHHM06], boolean function analysis [AA18], and many others. Using different complexity assumptions, advantages have also been studied and proven in other settings. To mention a few, assuming **BQP** hard processes cannot be simulated by poly-sized classical circuits, Molteni *et al.* show that Pauli strings and related objects are learnable by quantum computers with classical examples, but not by classical computers [MGD26]. Molteni *et al.* also show, assuming **BQP** is low in the polynomial hierarchy, that the problem of identification, instead of replication, in PAC learning with classical examples is hard [MMD26]. However, much of the focus of this collection of prior work either focuses explicitly on quantum computation, or a mix of quantum computation and examples, as opposed to isolating the advantage of quantum examples. Furthermore, not all of this prior work explicitly lies in the field of testing and learning, which this paper is the focus on.

More specifically, there is also the subsection of quantum learning theory that focuses on studying advantages in quantum learning and testing problems. We start by referring to a survey on quantum property testing by Montanaro [MW16]. Such property testing problems include entropy estimation [LW19], hypothesis testing, and closeness testing [CWZ25]² to name a few. In the context of distributional testing, Gilyen *et al.* were able to show that quantum examples gave a polynomial advantage over classical examples [GL20], so long as one also has access to the unitary circuit which creates this example. Other such papers have studied this access model, for instance Canonne *et al.* [CKO25], where the advantage, amongst other reasons, come from being able to conjugate the generating circuit. Our results differ from these in a few ways. First, though we assume an oracle setting, our separations do not rely on having access to the unitary which creates the quantum examples, meaning we cannot exploit the use of the circuit in our learning problems. Secondly, whilst these prior results study testing (of distributions), our separations hold for learning of distributions. Also of note is though we focus on distributions, there is also much work on learning and testing of functions,. Some such work includes juntas [TY26; AS07], monotonicity and triangle freeness [CNS26]. Of interest is the latter work of Caro *et al.*, who show that there exists a problem in function testing for which classical examples actually exhibit an exponential advantage over quantum examples. It would be interesting to see whether such a separation exists in learning as well.

Placing this work in the broader context of learning theory, this work has connections to PAC learning. One key question this work explores is what the differences between PAC learning and

²This manuscript is a collection of bounds provable by using the methods of [WZ25] and [TWZ25].

generating are. Kearns *et al.* [KMRRSS94] first studied this difference between learning and generating, and showed the first example of a concept class that could efficiently be PAC generated, but not PAC learned. Sweke *et al.* [SSHE21] then further developed this into a computation separation between classical generators and quantum generators. More broadly, this work also explores the fundamental task of *learning* a distribution from samples, instead of just generating it. This has been very thoroughly studied, and we refer to a chapter by Dianikolas for a survey [Dia16].

1.4 Outline

We provide an overview of PAC learning and the types of oracle access we will consider in Section 2. We then move to show that function hardness does not imply distributional hardness in PAC learning relative to an oracle in Section 3. Then, we show a separation between quantum examples and classical examples relative to an oracle in Section 4. We then conclude and provide some open problems in Section 5.

2 Preliminaries

We use standard asymptotic notation throughout, as well as the (slightly less standard) $\tilde{O}(\cdot)$, $\tilde{\Omega}(\cdot)$ notation to omit polylogarithmic factors in the argument. We write “u.a.r.” for “uniformly at random”, and will denote addition modulo 2 by $+$ instead of $\oplus$. Finally, the concatenation of two strings a and b is denoted $a||b$. In what follows, we use *computationally efficient* (or just *efficient*) to indicate that an algorithm runs in polynomial time (in the relevant parameters).

2.1 PAC Learning

We introduce the necessary definitions for PAC learning in this section. All definitions will be for general oracles, with more details on the specific forms of oracles we use given in Section 2.3. First, we recall the standard definition of PAC learning of functions: for in depth-survey of this field, we refer the reader to [Ser25].

Definition 2.1 (PAC Learning of Functions). Let $\mathcal{D}$ be a probability distribution over n -bit strings, and $\mathcal{F}_n$ be the set of all functions with domain in the n -bit strings. An algorithm $\mathcal{A}$ is an $(\epsilon, \delta, O, \mathcal{D})$ -PAC learner for a concept class $\mathcal{C} \subseteq \mathcal{F}_n$ if for all $c \in \mathcal{C}$, when given the oracle $O(c, \mathcal{D})$, which outputs an example from $c(x)$ with x drawn according to probability distribution $\mathcal{D}$, with probability at least $1 - \delta$, the learner $\mathcal{A}$ outputs a hypothesis $h \in \mathcal{F}_n$ such that

$$\Pr_{x \leftarrow \mathcal{D}} [h(x) \neq c(x)] \leq \epsilon.$$

We say such an algorithm is a *PAC-learner* if it is an $(\epsilon, \delta, O, \mathcal{D})$ -PAC learner for every ϵ, δ , and arbitrary (unknown) distribution $\mathcal{D}$. We note that while the general form of PAC learning requires that the learner can output such a hypothesis when given examples drawn from *any* (unknown) distribution $\mathcal{D}$, sometimes, we will be interested in the case where the examples are drawn from a specific pre-defined distribution $\mathcal{D}$: an algorithm only required to succeed in this case will be said to be a *PAC-learner with respect to the distribution $\mathcal{D}$* . We define the *sample complexity* of the learner $\mathcal{A}$ as the number of samples taken by the algorithm, and we call the learner *efficient* if it runs in time polynomial in $n, 1/\epsilon$, and $\log(1/\delta)$.

We will be specifically interested in the case of *uniform* PAC learners, that is, PAC-learners with respect to the uniform distribution. We will also be concerned about PAC GEN learning of distributions, defined below. First, we need the concept of (approximate) generator:

Definition 2.2 (Approximate generator). Fix a distance measure d between probability distributions, and $\epsilon > 0$. An algorithm is said to be a (d, ϵ) -generator for a probability distribution $\mathcal{D}$ if it is computationally efficient, and the distribution $\mathcal{D}'$ of its output satisfies $d(\mathcal{D}, \mathcal{D}') \leq \epsilon$.

Definition 2.3 (PAC GEN Learning of Distributions). Fix a distance measure d between probability distributions. A learning algorithm $\mathcal{A}$ is an (ϵ, δ, O, d) -PAC GEN-learner for a distribution class $\mathcal{C}$, if for all $\mathcal{D} \in \mathcal{C}$, when given oracle $O(\mathcal{D})$, which outputs a sample according to the distribution $\mathcal{D}$, with probability $1 - \delta$, the learner $\mathcal{A}$ outputs a (classical) (d, ϵ) -generator $GEN_{\mathcal{D}'}$ for $\mathcal{D}$. The generator $GEN_{\mathcal{D}'}$ has access to all the resources that the original learning algorithm $\mathcal{A}$ has, except the example oracle $O(\mathcal{D})$. Namely, if distribution class is induced by a function g encoded via an oracle, the outputted generator may still use calls to the oracle g . We further say the learning algorithm *efficient* if it outputs the generator in polynomial time.

The notion of distance we will adopt in this paper is the total variational (TV) distance, which we expand on in Section 2.2. We emphasize that the above definition provides the outputted generator $GEN_{\mathcal{D}'}$ with the same “resources” as the learning algorithm $\mathcal{A}$ uses, *except for the example oracle itself*. This is crucial in this paper, as our hard instances rely two oracles, $O(\mathcal{D})$ (the example oracle) and oracle access to some auxiliary function: the generating algorithm will still have access to the latter, but not the former.

Thus, Definition 2.1 is concerned with learning functions, while Definition 2.3 is concerned with learning distributions. Accordingly, we will refer to algorithms learning functions as *PAC learners*, or simply *learners*, and algorithms generating distributions as *PAC generators*, or simply *generators*. Further, notice that the definitions do not make any distinction between quantum and classical algorithms. For the purposes of this paper, we will assume that all learners have quantum computation. Hence, when we refer to “classical learners,” we mean “quantum algorithms with *classical examples*,” and “quantum learners” as “quantum algorithms with *quantum examples*.”

Finally, we also note that every boolean function on n variables naturally induces a distribution $D_f \in \mathcal{D}_{n+1}$, the set of distributions on $n + 1$ variables, via *distribution classes*.

Definition 2.4 (Distribution Class). Given a boolean function $f \in \mathcal{F}_n$, we define the distribution $D_f \in \mathcal{D}_{n+1}$ as the distribution that is uniform over $(n + 1)$ -bit strings of the form $x||f(x)$. Additionally, given a concept class $\mathcal{C} \subseteq \mathcal{F}_n$, we define the distribution class $\mathcal{D}_{\mathcal{C}} \subseteq \mathcal{D}_{n+1}$ as

$$\mathcal{D}_{\mathcal{C}} = \{D_c \mid c \in \mathcal{C}\}.$$

2.2 Total Variation Distance

In this section, we formally define and give properties on TV distance. Intuitively, one can think of the TV distance as the maximal difference between how two probability distributions assign probabilities to (sets of) events. We here focus on discrete domains, for simplicity, and as this is the focus of our work.

Definition 2.5 (TV Distance). Let $\mathcal{D}_1$ and $\mathcal{D}_2$ be two probability distributions defined over S . Then, the TV distance between $\mathcal{D}_1$ and $\mathcal{D}_2$ is given by

$$\text{TV}(\mathcal{D}_1, \mathcal{D}_2) = \sup_{E \subseteq S} (\mathcal{D}_1(E) - \mathcal{D}_2(E)) = \frac{1}{2} \sum_{x \in S} |\mathcal{D}_1(x) - \mathcal{D}_2(x)|.$$

One can check that the TV distance is a metric, and takes values in $[0, 1]$. We also use the following standard alternate form of the TV distance:

Lemma 2.1. *For any two probability distributions $\mathcal{D}_1, \mathcal{D}_2$ over S , we have*

$$\text{TV}(\mathcal{D}_1, \mathcal{D}_2) = 1 - \sum_{x \in S} \min(\mathcal{D}_1(x), \mathcal{D}_2(x)).$$

See, e.g., [MU05, Chapter 11] for more on total variation distance.

2.3 Oracle Definitions

In this section we provide more detail on what we mean by access to different types of examples. We will use three types of oracle access to the examples, which we define below.

Definition 2.6 (Oracles). The three oracles we are concerned about are the SAMPLE oracle, the quantum equivalent QSAMPLE oracle, and the PEX oracle. We note for all three oracles, each call is independent across executions.

1. Given a distribution $\mathcal{D}$, the SAMPLE oracle returns some x drawn from $\mathcal{D}$. The outputs across different calls of the oracle are independent.
2. Given a distribution $\mathcal{D}$, the QSAMPLE oracle returns a superposition of all elements x in the distribution, $\sum_x \sqrt{\mathcal{D}(x)}|x\rangle$.
3. Given a distribution $\mathcal{D}$ and a function f , the random example oracle PEX first samples x according to the distribution $\mathcal{D}$, then returns the tuple $(x, f(x))$. The outputs across different calls of the oracle are independent.

That is, the SAMPLE oracle returns a random sample drawn from the distribution $\mathcal{D}$, and the QSAMPLE oracle returns a “quantum example” from the same distribution, which is a superposition according to the weights of the distribution.³ On the other hand, the PEX oracle is an example oracle, which samples an input x according to the distribution $\mathcal{D}$, and returns the tuple consisting of the sample x and the evaluation $f(x)$.

2.4 One-way functions

Our first result will rely on the use of *one-way permutations*: in this subsection, we recall some cryptographic definitions and relevant background, and refer the interested reader to [Gol01, Chapter 1] for a thorough coverage. We start with the definition of a *one-way function*.

Definition 2.7 (One-way functions). A function $f: \{0, 1\}^n \rightarrow \{0, 1\}^n$ is a *one-way function* if:

1. It is *easy to compute*. That is, there exists a deterministic polynomial-time algorithm that on input x , outputs $f(x)$.

³In particular, measuring this state in the computational basis yields a classical sample from the distribution $\mathcal{D}$.

2. It is *hard to invert*. That is, for every probabilistic polynomial-time (PPT) algorithm $\mathcal{A}$, every polynomial $p(n)$ and all sufficiently large n ,

$$\Pr_{x \leftarrow \{0,1\}^n} [\mathcal{A}(1^n, f(x)) \in f^{-1}(f(x))] < \frac{1}{p(n)}.$$

That is, for most $y = f(x)$, no PPT algorithm can find any preimage of y with non-negligible probability.

A *one-way permutation* is then an injective one-way function. Given a one-way function, we can also define the related *hardcore predicate*.

Definition 2.8 (Hardcore Predicate). A polynomial computable predicate $b : \{0,1\}^* \rightarrow \{0,1\}$ is a *hardcore* of a function f if for every PPT algorithm A' , every polynomial $p(\cdot)$, and all sufficiently large n ,

$$\Pr_x[A'(f(x)) = b(x)] < \frac{1}{2} + \frac{1}{p(n)}.$$

We know of the existence of a specific form of hardcore predicate which we will use, for example, by [Gol01, Theorem 2.5.2]. Any one way function can be converted into the *hardcore* form.

Theorem 2.1. *Let f be a one-way function, and define $g(x, r)$ as the tuple $g(x, r) = (f(x), r)$, where r is a string such that $|x| = |r|$. Let $b(x, r)$ denote the inner product of the binary vectors x and r . Then the predicate b is a hardcore of the function g .*

It is worth noting the string r is not kept hidden from the adversary. We then have the following standard result, which shows that predicting a predicate implies inverting the one-way function: i.e., the predicate captures the computational hardness.

Theorem 2.2 (Goldreich-Levin, [GL89]). *If there exists a PPT algorithm that predicts $b(x, r)$ when given a tuple $(f(x), r)$ with probability $\frac{1}{2} + \eta$, then there exists an algorithm that inverts f with probability at least $\text{poly}(\eta)/\text{poly}(n)$, and runs in time $\text{poly}(n, 1/\eta)$. We refer to η as the advantage, and refer to it as non-negligible if it is at least $1/\text{poly}(n)$ in magnitude.*

Hence, if η non-negligible, there exists a PPT algorithm which can invert the hardcore predicate. We conclude the section by noting that it is not known whether one-way functions exist; in fact, showing their existence would imply $\mathbf{P} \neq \mathbf{NP}$. Furthermore, the existence of one-way functions alone also does not imply the existence of one-way permutations, and the latter is a stronger assumption. However, their existence is widely conjectured, and constitutes a standard cryptographic assumption.

3 Function Hardness Does Not Imply Distributional Hardness

3.1 Context Of Prior Work by Sweke *et al.*

In this section, we summarize a potential strategy given by Sweke *et al.* that might be used to prove a separation between quantum and classical PAC learning. One key result they were able to prove was that if a concept class $\mathcal{C}$ is efficiently PAC-learnable with respect to the uniform distribution, then the induced distribution class $\mathcal{D}_{\mathcal{C}}$ is also efficiently PAC-GEN learnable.

Theorem 3.1 (Function Learnability Implies Distribution Generation, [SSHE21]). *If a concept class $\mathcal{C}$ is efficiently classically (resp. quantumly) PAC learnable with respect to the uniform distribution and the PEX oracle, then the distribution class $\mathcal{D}_{\mathcal{C}}$ is efficiently classically (resp. quantumly) PAC GEN-learnable with respect to the SAMPLE oracle and TV distance.*

They left the reverse direction open as a conjecture.

Conjecture 3.1 (Function Hardness Implies Distribution Hardness, [SSHE21]). *If a concept class $\mathcal{C}$ is not efficiently classically (quantumly) PAC learnable with respect to the uniform distribution and the PEX oracle, then the distribution class $\mathcal{D}_{\mathcal{C}}$ is not efficiently classically (quantumly) PAC GEN-learnable with respect to the SAMPLE oracle and TV distance.*

The hope was that if this conjecture was true, and one could find a boolean concept class $\mathcal{C}$ such that:

1. $\mathcal{C}$ is not efficiently classically PAC learnable, with respect to the uniform distribution and PEX oracle, and
2. $\mathcal{C}$ is efficiently quantumly learnable, with respect to the uniform distribution, and the PEX oracle,

then this would immediately imply that the induced distribution class $\mathcal{D}_{\mathcal{C}}$ would not be efficiently classically PAC GEN-learnable, yet would be efficiently quantumly PAC GEN-learnable. Though it is currently not known whether a concept class does satisfy the above two criteria, proving the reverse direction of this conjecture true would allow one to instantly leverage such a concept class, if it exists, into a PAC GEN-learning separation. Then, should this hold true, Sweke *et al.* conjectured that one could potentially modify both of the above results to prove hardness separations for classical and quantum *samples*, instead of classical and quantum computation separations, for boolean concept classes and their induced distributions.

In the remainder of this section, we establish Theorem 1.2 (formally restated as Theorem 3.2), providing strong evidence that this reverse direction does not hold: i.e., we construct an explicit counterexample showing this conjecture to be false relative to an oracle.

3.2 A Counterexample to Conjecture 3.1

Assuming the existence of one-way permutations, we show Conjecture 3.1 to be false by explicitly constructing a function class which is not efficiently learnable, but induces a distribution that is efficiently PAC-gen learnable (and in fact, exactly so). First, note given any function $f(x)$, one can naturally define a distribution class given by the tuples $(x, f(x))$, via Definition 2.4. We thus create a distribution class consisting of a family of functions $\{f(x)\}$, which then induces a family of distributions over tuples $\{(x, f(x))\}$, and show whilst the former cannot be efficiently learned, the latter can be efficiently generated.

Theorem 3.2. *Assume one-way permutations exist, and let $\mathcal{W}$ be a (classical) oracle providing query access to a one-way permutation. Then there is a concept class $\mathcal{C}$ that can be efficiently generated with respect to the SAMPLE oracle and TV-distance, but is not efficiently learnable with respect to the uniform distribution and the PEX oracle.*

We make a slight alteration to the PAC learning requirement in Definition 2.1, strengthening the outputted hypothesis. Recall in the classical definition, the learner must output a hypothesis $h(x)$ that only errors with probability ϵ . However, we will allow the learner to output a function $h(\mathcal{W}, x)$ that can be efficiently evaluated. In other words, the learner can output a circuit, which in addition to the input, can make a polynomial number of calls to the permutation oracle, with the aim that this circuit only errors with probability ϵ . The reason for doing so is that our PAC GEN learner relies on access to this permutation oracle to be efficient. If we did not give the hypothesis function the same type of access, one might argue that the separation is because the generator has access to additional resources, being oracle queries in its generating process, whilst the hypothesis output does not. In this case, we allow the learner to output something comparable, and prove that even if the learner is able to output a hypothesis which can incorporate a polynomial amount of oracle calls, Theorem 3.2 still holds. We now move to the proof.

Proof. Notice all queries are classical under the given oracle access. We will show the result for classical computers, and the argument for quantum computers is identical (as discussed in the remarks after the proof). We let $g : \{0, 1\}^{n-1} \rightarrow \{0, 1\}^{n-1}$ be a one-way permutation to which the algorithms have oracle access. In other words, there is an oracle which the algorithm can query at any time with a string x , and the oracle will return the mapping of x under the one-way permutation $g(x)$. Next, let $r \in \{0, 1\}^{n-1}$ be a uniformly sampled nonzero vector, and let $B_r(x) : \{0, 1\}^{n-1} \rightarrow \{0, 1\}$ be the function $B_r(x) = \langle x, r \rangle \bmod 2$, the hardcore predicate of the function $l(x) = (g(x), r)$. Note that, by Theorem 2.2, being given l and inverting B_r is equivalent to inverting g . Given a specified one-way permutation g , and a randomly sampled r , we define our boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ as follows. For ease of representation, instead of taking the input as an n -bit string, we will break the input into two strings $t \in \{0, 1\}^{n-1}$ and $b \in \{0, 1\}$, so that $x = t||b$. Then f is given by:

$$f(x) = f(t||b) = \begin{cases} B_r(t) = \langle t, r \rangle \bmod 2, & \text{if } b = 0, \\ B_r(g^{-1}(t)) = \langle g^{-1}(t), r \rangle \bmod 2, & \text{if } b = 1. \end{cases}$$

Then, given a one-way permutation g , our concept class $\mathcal{C}$ is all such f obtained from the one-way permutation g and the 2^{n-1} possible random strings r . The corresponding distribution generated is the uniform one over strings $x||f(x)$.

First, notice that in both the learning and generating setting, the algorithm can efficiently learn the hidden string r : indeed, to do so, it suffices to find $n - 1$ linearly independent nonzero strings t with $b = 0$ in their random samples, and then solve a system of equations; with overwhelming probability, the former will occur in a polynomial amount of samples. However, we argue that even though learning r is easy, it is hard to learn the function f . To see this, assume an algorithm was able to predict $f(x)$ with non-negligible advantage. Then, they must be able to predict $f(x)$ for strings where $b = 1$ with non-negligible advantage. Under the PEX oracle given to the learner, the algorithm is provided with an example $g(t)$ by taking the first $n - 1$ bits of x . Combined with having already learned r , this gives them a tuple $(g(t), r)$. Predicting $f(x)$ with advantage in this case means that they can predict $B_r(g^{-1}(t))$ with advantage. To see why, assume an algorithm was able to predict f with non-negligible advantage over a uniformly random choice of $x = t||b$. Then, as we will make rigorous shortly, they must in particular be able to predict $f(x)$ for strings of the form $x = t||1$ (i.e., with $b = 1$), which, even knowing r , means predicting with non-negligible advantage the hardcore predicate $\langle s, r \rangle$ of g as defined in Theorem 2.1, given a tuple $(g(s), r)$ (where

$s := g^{-1}(t)$). – contradicting that g is a one-way permutation. (Note that even a polynomial number of samples from the permutation oracle $\mathcal{W}$ would still not help, unless by chance the preimage of t was in these samples: which only happens with probability $o(1)$, where the $o(1)$ term decays exponentially in n .) We now proceed to formalize this intuition.

The fact that g is a one-way permutation implies, for any efficient hypothesis $h(\mathcal{W}, x)$ that the learner produces, the probability that it agrees with the hardcore predicate is bounded by $\Pr_x[h(\mathcal{W}, x) = b(x)] < \frac{1}{2} + \frac{1}{p(n)}$ for every $p(n)$. The hypothesis is required to predict the hardcore predicate half of the time, where the bit string ends in 1. Hence, we get that

$$\Pr[h(\mathcal{W}, x) = f(x)] \leq \frac{3}{4} + \frac{1}{p(n)},$$

where the probability is taken over the uniform distribution. As such, this cannot be efficiently PAC learned for $\epsilon < 0.25$.

However, notice the generator has an effective way of generating this distribution, and in fact, it can generate the *exact* distribution. First, it learns r . Second, it samples the random strings t and b . If $b = 0$, it computes $B_r(t)$, having learned the string r . Then, it can output $t||0||B_r(t)$ as a sample. Otherwise, if $b = 1$, it can feed the string t into the one-way permutation oracle, to obtain the string $s := g(t)$: it can now compute $B_r(t) = B_r(g^{-1}(s))$, and output the string $s||1||B_r(t)$. Since g is a one-way permutation, uniformly sampling t is equivalent to uniformly sampling s . The learning algorithm just needs to learn r (which, as outlined above, can be done with high probability using a polynomial amount of queries) before outputting a generator GEN which samples from the distribution in constant time (and at most one call to the permutation oracle) as described above. This proves the theorem for classical computers. $\square$

We make three concluding remarks here. First, note that even when given quantum computation, one-way functions are still assumed to be hard to invert. As such, the above results also generalize to hold even for quantum algorithms. Furthermore, while our construction only assumes classical access to the one-way permutation, the results also hold when the generating algorithms are given *both* quantum and classical access to the one-way permutation (since one-way permutations are still assumed to be hard to invert given this type of access). By quantum access, we mean being given access to a uniform superposition $\sum_x |x\rangle|g(x)\rangle$ over the output of g . Note though, as part of our construction, the generating algorithm needs to explicitly evaluate a uniformly sampled input in the one-way permutation, and thus requires at least classical access: having *only* quantum access to the one-way permutation does not suffice for our argument. Finally, we explicitly need one-way permutations, and not just one-way functions, since we need the preimage of a uniformly random element to also be uniformly distributed. We leave it as an open question to relax this requirement to only needing one-way functions, or indeed a separate construction that does not require this at all.

4 A Quantum/Classical Separation in PAC GEN Learning

After having provided strong evidence that Conjecture 3.1 is false in the previous section, we now establish a separation between quantum and classical examples. At a high-level, our approach for this second separation is as follows. First, we sample a random boolean function g , afterwards available to the algorithm as an oracle. Then, we uniformly at random sample a

nonzero vector a , and create a function f with period a by defining $f(x) = g(x) + g(x + a)$, where $+$ is bitwise XOR (bitwise mod 2 addition). Now, while f clearly has a as period, it might also have pseudoperiods,⁴ and is not guaranteed to be perfectly 2-to-1. Nevertheless, we show that the number of pseudoperiods in this function is low (with high probability), which as a result means we can still use Simon’s algorithm to recover the period with high probability [KLLN16]. We proceed with formal definitions and proofs now.

4.1 Probably Approximate Simon’s (PAS) Functions

We begin by defining the main building block of our concept class.

Definition 4.1 (Probably Approximate Simon’s Function). Let $g : \{0, 1\}^n \rightarrow \{0, 1\}^n$ be a u.a.r function, and a be a uniformly sampled random vector. We can construct a function f by imposing g onto itself via a shift of a , $f(x) = g(x) + g(x + a)$. We call such a function f a *probably approximate Simon’s function* (PAS function).

As previously discussed, this function is guaranteed to have period a (i.e., $f(x) = f(x + a)$ for every x), but may also contain more collisions of this form for specific x . We can obtain a measure of these unwanted collisions via the *pseudoperiod factor*.

Definition 4.2 (Pseudoperiod Factor). Given a PAS function with period a , the *pseudoperiod factor* $\varepsilon(f, a)$ is the quantity

$$\varepsilon(f, a) = \max_{t \in \{0, 1\}^n \setminus \{0, a\}} \Pr_x[f(x) = f(x + t)]. \quad (4.1)$$

We also say that input x admits a pseudoperiod t if $f(x) = f(x + t)$.

This parameter was introduced by Kaplan *et al.* [KLLN16] to quantify how far a function is from being perfectly 2-to-1. We will prove the following.

Lemma 4.1. *Let $g : \{0, 1\}^n \rightarrow \{0, 1\}^n$ be a u.a.r. function. For each nonzero string a , define f_a by $f_a(x) = g(x) + g(x + a)$ for all $x \in \{0, 1\}^n$. Then, with probability $1 - o(1)$ over the choice of g , for all nonzero a , we have that $\varepsilon(f_a, a) = O(\frac{n}{2^n \log n})$.*

We will prove this lemma at the end of the subsection; before this, we introduce the definitions and results necessary to the proof. First, observe that if a given input x admits a pseudoperiod t , we can rewrite this in terms of the original random function g : since $f(x) = f(x + t)$, we have that $g(x) + g(x + a) = g(x + t) + g(x + t + a)$, or equivalently, that $g(x) + g(x + a) + g(x + t) + g(x + t + a) = 0$. Setting $\Delta_x := g(x) + g(x + t) + g(x + a) + g(x + t + a)$, admitting a pseudoperiod is equivalent to having $\Delta_x = 0$. Further, notice that this now defines a quadrilateral across the boolean cube with edges a and t , illustrated in Fig. 3. Furthermore, any two quadrilaterals with differing x values are either completely disjoint or coincide completely on the same face, meaning this partitions the boolean cube into 2^{n-2} distinct faces.

We now follow the analysis of Daemen and Rijmen [DR07], who derived concentration bounds on ε for singular shifts (i.e., the probability that for a random function, $g(x) = g(x + t)$). We proceed to adapt this analysis for double shifts.

⁴A pseudoperiod for an input x (and an a -periodic function f) is any string $t \neq a$ such that $f(x) = f(x + t)$.

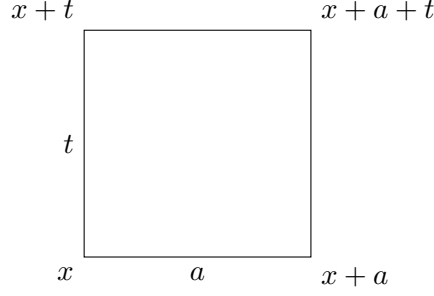


Figure 3: The quadrilateral spanned by the pseudoperiod t and the period a .

Definition 4.3 (Cardinality of the differential). For a fixed g , a and t , the *cardinality of the differential* $N(g, a, t)$ is the number of faces with edges a and t such that g sums to zero across the vertices:

$$N(g, a, t) = |\{x, x+a, x+t, x+a+t\} \mid g(x) + g(x+a) + g(x+t) + g(x+a+t) = 0\}|.$$

Note that $N(g, a, t) = \frac{|\Delta_x|}{4}$, where $|\Delta_x|$ is the count of the number of inputs x such that $\Delta_x = 0$. This is because for each face that sums to zero, there are four inputs x for which $\Delta_x = 0$. We can show the following about the distribution of $N(g, a, t)$.

Lemma 4.2. For fixed a, t and $g : \{0, 1\}^n \rightarrow \{0, 1\}^n$ chosen u.a.r., $N(g, a, t)$ is a random variable with binomial distribution:

$$\Pr[N(g, a, t) = i] = (2^{-n})^i (1 - 2^{-n})^{2^{n-2}-i} \binom{2^{n-2}}{i}, \quad 0 \leq i \leq 2^{n-2}.$$

That is, $N(g, a, t) \sim \text{Bin}(2^{n-2}, 2^{-n})$.

Proof. A random function maps the 2^n different input values x to independent output values $g(x)$, and hence for each quadrilateral $\{x, x+a, x+a+t, x+t\}$, it maps the output over the vertices of the quadrilateral to independent values too, and thus the sum of outputs to independent values. Each of these mappings is successful if it maps to exactly 0, which happens with probability $\frac{1}{2^n}$, and we have 2^{n-2} mappings, corresponding to the number of distinct quadrilaterals. $\square$

We can now complete the proof of Lemma 4.1.

Proof of Lemma 4.1. For a fixed t , by Lemma 4.2, we know that $N(g, a, t)$ has distribution $Y \sim \text{Bin}(2^{n-2}, 2^{-n})$, and thus has expectation $\mu = 1/4$. By standard Chernoff bounds and setting $(1 + \delta)\mu = 4m$

$$\begin{aligned} \Pr[Y \geq (1 + \delta)\mu] &\leq \left(\frac{e^\delta}{(1 + \delta)^{1+\delta}} \right)^\mu \\ \Pr[Y \geq 4m] &\leq \left(\frac{e^{4m-1}}{(4m)^{4m}} \right)^{1/4} \\ &= e^{m-1/4} (4m)^{-4m} \end{aligned}$$

$$\leq \left(\frac{e}{4m}\right)^m.$$

Let $T = 2^n$. We note there are at most T possible periods a , and for each of these periods a , at most T possible pseudoperiods. This means there are at most T^2 pairs of period and possible pseudoperiod. Let m be the number of “bad” trials, in the sense that a is a period and t is a pseudoperiod (as opposed to only a possible one). By standard Chernoff bounds, we have that

$$\Pr[\max_t Y_t \geq m] \leq \Pr\left[\bigcup_{t=1}^T \{Y_t \geq m\}\right] \leq T \Pr[Y \geq m] \leq T \left(\frac{e}{4m}\right)^m.$$

We can now choose the rightmost value to be small. Choose $m = \frac{8 \log T}{\log \log T}$. Then, we have

$$\begin{aligned} \log[T(e/4m)^m] &= 2 \log T - m \log(4m/e) \\ &= \log T - m(\log \log T - \log \log \log T + \log(32/e)) \\ &\leq 2 \log T - m(0.5 \log \log T) \\ &\leq -2 \log T. \end{aligned}$$

This implies that $\Pr[\max_t Y_t \geq m] \leq T^{-2}$, and hence, we have that $\max_t N(g, a, t) \leq O(n/\log n)$ with probability $1 - T^{-2}$. Then, for each a , dividing by the number of possible x values, we get that $\varepsilon(f, a) = O\left(\frac{n}{2^n \log n}\right)$, with probability $1 - 2^{-2n} = 1 - o(1)$. $\square$

4.2 Hardness Construction

We will make use of the analysis of the Approximate Simon’s Promise problem by Kaplan *et al.* to show that Simon’s algorithm for sufficiently small pseudoperiods will still return a correct period with high probability.

Theorem 4.1 (Theorem 1, [KLLN16]). *If $\varepsilon(f, a) \leq p_0 < 1$, then Simon’s algorithm returns a with cn queries with probability at least $1 - (2(\frac{1+p_0}{2})^c)^n$, where $c > 0$ is a parameter capturing how many times the algorithm is run to boost probability.*

Let $g: \{0, 1\}^n \rightarrow \{0, 1\}^n$ be a function encoded via an oracle. This function serves as the base of our concept class $\mathcal{C}$, which we now describe how to construct.

1. For each nonzero string a , define the corresponding PAS function constructed from g . There will be $2^n - 1$ of these.
2. Each of these PAS functions f_a induces a distribution over strings $x || f_a(x)$, with x uniform over n -bit strings.
3. These $2^n - 1$ distributions form the concept class $\mathcal{C}$.

One of these distributions will be picked as the one that needs to be PAC-generated, with one learner given classical SAMPLE access, and one given quantum QSAMPLE access. We then show the following.

Theorem 4.2. *For the concept class $\mathcal{C}$ described above, with additional access to the function g encoded via oracle $\mathcal{W}$, there is a quantum algorithm with QSAMPLE access to f that can efficiently PAC-generate this concept class. However, no quantum algorithm with only SAMPLE access to f can efficiently PAC-generate this concept class.*

Proof. We first outline how this can efficiently be done with quantum examples. First, we choose a u.a.r. function g such that

$$\varepsilon(f_a, a) = O\left(\frac{n}{2^n \log n}\right)$$

holds simultaneously for all non-zero a , which is guaranteed to exist by Lemma 4.1 (and the probabilistic method). We use this function g to define our concept class $\mathcal{C}$ as described above. Then, choosing one such f in this concept class, given uniform superposition examples from QSAMPLE, and for some appropriately large constant c , one can run Simon’s algorithm via Theorem 4.1 to obtain the period a . This succeeds with probability $(1 - (2(\frac{1+p_0}{2})^c)^n)$ and requires cn samples. Then, conditional on obtaining the period a , the distribution can now be generated from exactly: this is done by uniformly sampling x , then querying g on inputs x and $x + a$, to calculate $f(x) = g(x) + g(x + a)$ before outputting $x || f(x)$. (Observe that although the generating algorithm still requires oracle access to the function g , crucially it no longer needs access to the example oracle for f , meaning it can successfully PAC generate this distribution class.) This algorithm only errs if Simon’s algorithm fails to find the period a : the probability of failure can be made arbitrarily small by increasing the constant c .

We now argue that with only *classical* SAMPLE examples, even a quantum algorithms cannot generate this distribution efficiently. First, we note that obtaining the period a with classical samples requires exponential sample complexity: this follows from the classical *query* bound of $\Omega(\sqrt{2^n})$ due to Cleve [Cle11], meaning any classical algorithm requires at least that many queries to uncover the period a .⁵ Then, any quantum learner, if it only has classical examples, can be simulated (possibly very inefficiently) by a classical computer with same sample complexity: this possibly computationally inefficient simulation yields a classical algorithm to which the query lower bound applies.

Having shown that finding a cannot be done efficiently, it is sufficient to argue that, without learning a , no generator can sample from a distribution close to the required one. To do so, the first thing we argue is that a classical learner does not gain too much information from the guarantee that the random function g will create a concept class where every f has a small pseudoperiod factor. This is done by observing that via Lemma 4.1, any random function g satisfies this with probability $1 - o(1)$, so even if the classical algorithm were to condition on such random functions, they would not meaningfully reduce the search space.

Assume the generator produces samples from a distribution $\mathcal{D}$, and the true distribution is $\mathcal{D}_f$. We want to bound the TV distance between the two distributions. Assume that the learning algorithm sees $q = \text{poly}(n)$ examples, and (as not charging duplicate samples to the algorithm only strengthens the lower bound) assume they are distinct. We can assume without loss of generality that the distribution that the algorithm generates from $\mathcal{D}$ is uniform over inputs x . In other words, the algorithm uniformly at random samples x , and tries to guess $f(x)$. (This is without loss of

⁵We note that there is a slight technicality, with this being a query bound, whereas in this setting we are using classical examples. However, the former is strictly stronger than the latter, since in the query setting, an algorithm can choose which x to query, whereas in the generating setting the SAMPLE oracle randomly gives examples, taking choice away from the algorithm.

generality because the real distribution $\mathcal{D}_f$ is uniform over x , so having $\mathcal{D}$ not uniform only increases TV distance.) For the sake of argument, we will assume that if $x+a$ is in the queries, the algorithm will always succeed, by being able to query g on both points, and thus reconstruct f (this is only beneficial to the algorithm). Else, with no knowledge of $x+a$, $f(x)$ behaves like a uniformly random n -bit string, and the learner can only guess it with probability $\frac{1}{2^n}$. Hence, the number of points on which $\mathcal{D}_f$ and $\mathcal{D}$ agree is $\text{poly}(n)$ with high probability, and hence the probability mass on which they agree is with high probability $\frac{\text{poly}(n)}{2^n}$. Then, the TV distance is bounded by:

$$\begin{aligned} \text{TV}(\mathcal{D}, \mathcal{D}_f) &= 1 - \sum_x (\mathcal{D}(x) \wedge \mathcal{D}_f(x)) \\ &\geq 1 - \frac{\text{poly}(n)}{2^n} \\ &= 1 - 2^{-\Omega(n)}. \end{aligned}$$

Overall, with high probability, if the learner does not know a , it will generate samples from a distribution that has TV distance close to 1, showing that no quantum algorithm with only SAMPLE access can efficiently PAC-generate this concept class. $\square$

Remark 4.1. We note that embedding the period in such a way that can be recovered via Simon’s algorithm is crucial. This is because in order to require the period, Simon’s algorithm only requires uniform superpositions of f . A stronger result might be to try using Boneh and Lipton’s period embedding, as described by Zhandry in [Zha21]. Indeed, this would create such a separation using all quantum secure functions. However, crucially Boneh and Lipton’s period embedding algorithm requires the ability to do more than just uniform superpositions. Among other things, the learner must be able to evaluate f at specific points via classical queries (see, Section 6 in [BL95]), which cannot be done in this model, since QSAMPLE does not actually give query access, and only gives the uniform superposition example. An attempt to generalize this separation must only rely on querying f , or the induced distribution in uniform superposition.

5 Conclusion

We gave an explicit distribution class that, relative to an oracle, can be efficiently generated by a quantum algorithm with quantum examples, but not by a quantum algorithm with only classical examples. To the best of our knowledge, this is the first separation between quantum and classical examples shown in the learning setting. We conclude with a few avenues for future research.

Oracles. In both our constructions for Theorem 3.2 and Theorem 4.2, we define the PAC learning instance with regards to an auxiliary function, which induces the concept and distribution classes. We then allow both the learner and the outputted generator to query this auxiliary function by providing oracle access to it. It would be interesting to see if a hard instance could be constructed in either of these instances without having the concept or distribution classes being induced by this auxiliary function. Or as an intermediate, it would be interesting to see if hard instances could be constructed where though the concept and distribution classes are induced by this auxiliary function, the outputted generators do not require oracle access to this function. On the other hand, we think that studying what oracle separations can be proven for learning and testing is also an interesting and underexplored area that may be worth pursuing.

Relaxed Assumptions for Theorem 3.2. In our proof of Theorem 3.2, we assume the existence of one-way permutations. Though this is a standard assumption in cryptography, we wonder if such a strong assumption is necessary. In other words, it would be interesting to see if a hard instance that relies on weaker assumptions could be constructed. For example, is there a hard instance that only requires assuming the existence of one-way functions, instead of one-way permutations? Or is there a hard instance that does not rely on one-way functions, but possibly another cryptographic primitive, such as trapdoor functions.

Maximal Quantum and Classical Example Separations. Implicitly, in the proof of Theorem 4.2, we show that there exists a distribution class that can be generated in $O(n)$ quantum examples, but cannot be generated with less than $\Omega(2^{n/2})$ classical examples. This result just follows directly from the sample bounds in Simon’s problem. It would be interesting to see what the maximal separation between quantum examples and classical examples in this setting could be. For example, Aaronson and Ambainis [AA18] show that in regards to the sample complexity for boolean functions, the forrelation problem can be solved in 1 quantum query, but requires $\tilde{\Omega}(2^{n/2})$ classical queries. It would be interesting to see if such a separation between quantum and classical examples in the learning setting could be shown as well.

Acknowledgements. We would like to thank Clément Canonne for comments, feedback and proofreading for this writeup and for discussions on the work. We would like to thank Ryan Sweke for discussions on their previous work, comments on this work, and for pointing to [GL20]. We would like to thank Matthias Caro for comments on this work, and for pointing us to [MMD26] and [MGD26].

References

- [AA18] Scott Aaronson and Andris Ambainis. “Forrelation: A Problem That Optimally Separates Quantum from Classical Computing”. In: *SIAM J. Comput.* 47.3 (2018), pp. 982–1038. DOI: [10.1137/15M1050902](https://doi.org/10.1137/15M1050902).
- [AS07] Alp Atici and Rocco A. Servedio. “Quantum Algorithms for Learning and Testing Juntas”. In: *Quantum Inf. Process.* 6.5 (2007), pp. 323–348. DOI: [10.1007/S11128-007-0061-6](https://doi.org/10.1007/S11128-007-0061-6).
- [AW18] Srinivasan Arunachalam and Ronald de Wolf. “Optimal Quantum Sample Complexity of Learning Algorithms”. In: *J. Mach. Learn. Res.* 19 (2018), 71:1–71:36.
- [BJ99] Nader H. Bshouty and Jeffrey C. Jackson. “Learning DNF over the Uniform Distribution Using a Quantum Example Oracle”. In: *SIAM J. Comput.* 28.3 (1999), pp. 1136–1153. DOI: [10.1137/S0097539795293123](https://doi.org/10.1137/S0097539795293123).
- [BL95] Dan Boneh and Richard J. Lipton. “Quantum Cryptanalysis of Hidden Linear Functions (Extended Abstract)”. In: *Advances in Cryptology - CRYPTO ’95, 15th Annual International Cryptology Conference, Santa Barbara, California, USA, August 27-31, 1995, Proceedings*. Vol. 963. Lecture Notes in Computer Science. Springer, 1995, pp. 424–437. DOI: [10.1007/3-540-44750-4_34](https://doi.org/10.1007/3-540-44750-4_34).

- [CKO25] Clément L. Canonne, Robin Kothari, and Ryan O’Donnell. “Uniformity Testing When You Have the Source Code”. In: *20th Conference on the Theory of Quantum Computation, Communication and Cryptography, TQC 2025, Indian Institute of Science, Bengaluru, India, September 15-19, 2025*. Ed. by Bill Fefferman. Vol. 350. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025, 7:1–7:20. DOI: [10.4230/LIPICS.TQC.2025.7](https://doi.org/10.4230/LIPICS.TQC.2025.7).
- [Cle11] Richard Cleve. *Classical lower bounds for simon’s problem*. 2011.
- [CNS26] Matthias C. Caro, Preksha Naik, and Joseph Slote. “Testing Classical Properties from Quantum Data”. In: *17th Innovations in Theoretical Computer Science Conference, ITCS 2026, Bocconi University, Milan, Italy, January 27-30, 2026*. Ed. by Shubhangi Saraf. Vol. 362. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2026, 34:1–34:26. DOI: [10.4230/LIPICS.ITCS.2026.34](https://doi.org/10.4230/LIPICS.ITCS.2026.34).
- [CWZ25] Kean Chen, Qisheng Wang, and Zhicheng Zhang. *A List of Complexity Bounds for Property Testing by Quantum Sample-to-Query Lifting*. 2025. arXiv: [2512.01971](https://arxiv.org/abs/2512.01971) [quant-ph].
- [DHHM06] Christoph Dürr, Mark Heiligman, Peter Høyer, and Mehdi Mhalla. “Quantum Query Complexity of Some Graph Problems”. In: *SIAM J. Comput.* 35.6 (2006), pp. 1310–1328. DOI: [10.1137/050644719](https://doi.org/10.1137/050644719).
- [Dia16] Ilias Diakonikolas. “Learning Structured Distributions”. In: *Handbook of Big Data*. Chapman and Hall/CRC, 2016, pp. 267–283. DOI: [10.1201/B19567-21](https://doi.org/10.1201/B19567-21).
- [DR07] Joan Daemen and Vincent Rijmen. “Probability distributions of correlation and differentials in block ciphers”. In: *J. Math. Cryptol.* 1.3 (2007), pp. 221–242. DOI: [10.1515/JMC.2007.011](https://doi.org/10.1515/JMC.2007.011).
- [GL20] András Gilyén and Tongyang Li. “Distributional Property Testing in a Quantum World”. In: *11th Innovations in Theoretical Computer Science Conference, ITCS 2020, Seattle, Washington, USA, January 12-14, 2020*. Ed. by Thomas Vidick. Vol. 151. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020, 25:1–25:19. DOI: [10.4230/LIPICS.ITCS.2020.25](https://doi.org/10.4230/LIPICS.ITCS.2020.25).
- [GL89] Oded Goldreich and Leonid A. Levin. “A Hard-Core Predicate for all One-Way Functions”. In: *Proceedings of the 21st Annual ACM Symposium on Theory of Computing, May 14-17, 1989, Seattle, Washington, USA*. ACM, 1989, pp. 25–32. DOI: [10.1145/73007.73010](https://doi.org/10.1145/73007.73010).
- [Gol01] Oded Goldreich. *The Foundations of Cryptography - Volume 1: Basic Techniques*. Cambridge University Press, 2001. ISBN: 0-521-79172-3. DOI: [10.1017/CB09780511546891](https://doi.org/10.1017/CB09780511546891).
- [Ham25] Yassine Hamoudi. “A Brief Introduction to Quantum Query Complexity”. In: *CoRR* abs/2508.08852 (2025). DOI: [10.48550/ARXIV.2508.08852](https://doi.org/10.48550/ARXIV.2508.08852). arXiv: [2508.08852](https://arxiv.org/abs/2508.08852).
- [KLLN16] Marc Kaplan, Gaëtan Leurent, Anthony Leverrier, and María Naya-Plasencia. “Breaking Symmetric Cryptosystems Using Quantum Period Finding”. In: *Advances in Cryptology - CRYPTO 2016 - 36th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 14-18, 2016, Proceedings, Part II*. Vol. 9815. Lecture Notes in Computer Science. Springer, 2016, pp. 207–237. DOI: [10.1007/978-3-662-53008-5_8](https://doi.org/10.1007/978-3-662-53008-5_8).

- [KMRRSS94] Michael J. Kearns, Yishay Mansour, Dana Ron, Ronitt Rubinfeld, Robert E. Schapire, and Linda Sellie. “On the learnability of discrete distributions”. In: *Proceedings of the Twenty-Sixth Annual ACM Symposium on Theory of Computing, 23-25 May 1994, Montréal, Québec, Canada*. ACM, 1994, pp. 273–282. DOI: [10.1145/195058.195155](#).
- [LW19] Tongyang Li and Xiaodi Wu. “Quantum Query Complexity of Entropy Estimation”. In: *IEEE Trans. Inf. Theory* 65.5 (2019), pp. 2899–2921. DOI: [10.1109/TIT.2018.2883306](#).
- [MGD26] Riccardo Molteni, Casper Gyurik, and Vedran Dunjko. “Exponential quantum advantages in learning quantum observables from classical data”. eng. In: *npj quantum information* 12.1 (2026), pp. 19–10. ISSN: 2056-6387.
- [MMD26] Riccardo Molteni, Simon Marshall, and Vedran Dunjko. “Quantum machine learning advantages beyond hardness of evaluation”. In: *International Conference on Learning Representations*. Ed. by C. Vondrick, B. Hariharan, C. Raffel, L. Pinto, D. Yang, and A. Faust. Vol. 2026. 2026, pp. 148258–148295.
- [MU05] Michael Mitzenmacher and Eli Upfal. *Probability and Computing: Randomized Algorithms and Probabilistic Analysis*. Cambridge University Press, 2005. ISBN: 978-0-521-83540-4. DOI: [10.1017/CB09780511813603](#).
- [MW16] Ashley Montanaro and Ronald de Wolf. “A Survey of Quantum Property Testing”. In: *Theory Comput.* 7 (2016), pp. 1–81. DOI: [10.4086/TOC.GS.2016.007](#).
- [RT22] Ran Raz and Avishay Tal. “Oracle Separation of BQP and PH”. In: *J. ACM* 69.4 (2022), 30:1–30:21. DOI: [10.1145/3530258](#).
- [Ser25] Rocco A. Servedio. “The Probably Approximately Correct Learning Model in Computational Learning Theory”. In: *CoRR* abs/2511.08791 (2025). DOI: [10.48550/ARXIV.2511.08791](#). arXiv: [2511.08791](#).
- [Sho97] Peter W. Shor. “Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer”. In: *SIAM J. Comput.* 26.5 (1997), pp. 1484–1509. DOI: [10.1137/S0097539795293172](#).
- [Sim97] Daniel R. Simon. “On the Power of Quantum Computation”. In: *SIAM J. Comput.* 26.5 (1997), pp. 1474–1483. DOI: [10.1137/S0097539796298637](#).
- [SSG24] Wilfred Salmon, Sergii Strelchuk, and Tom Gur. “Provable Advantage in Quantum PAC Learning”. In: *The Thirty Seventh Annual Conference on Learning Theory, June 30 - July 3, 2023, Edmonton, Canada*. Vol. 247. Proceedings of Machine Learning Research. PMLR, 2024, pp. 4487–4510.
- [SSHE21] Ryan Sweke, Jean-Pierre Seifert, Dominik Hangleiter, and Jens Eisert. “On the Quantum versus Classical Learnability of Discrete Distributions”. In: *Quantum* 5 (2021), p. 417. DOI: [10.22331/Q-2021-03-23-417](#).
- [TWZ25] Ewin Tang, John Wright, and Mark Zhandry. “Conjugate queries can help”. In: *CoRR* abs/2510.07622 (2025). DOI: [10.48550/ARXIV.2510.07622](#). arXiv: [2510.07622](#).

- [TY26] Avishay Tal and Weiqiang Yuan. “Quantum Advantage in Tolerant Junta Testing”. In: *41st Computational Complexity Conference, CCC 2026, Lisbon, Portugal, August 3-6, 2026*. Ed. by Dana Moshkovitz. Vol. 383. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2026, 34:1–34:17. DOI: [10.4230/LIPICS.CCC.2026.34](https://doi.org/10.4230/LIPICS.CCC.2026.34).
- [Val84] Leslie G. Valiant. “A Theory of the Learnable”. In: *Commun. ACM* 27.11 (1984), pp. 1134–1142. DOI: [10.1145/1968.1972](https://doi.org/10.1145/1968.1972).
- [WZ25] Qisheng Wang and Zhicheng Zhang. “Quantum Lower Bounds by Sample-to-Query Lifting”. In: *SIAM J. Comput.* 54.5 (2025), pp. 1294–1334. DOI: [10.1137/24M1638616](https://doi.org/10.1137/24M1638616).
- [Zha21] Mark Zhandry. “How to Construct Quantum Random Functions”. In: *J. ACM* 68.5 (2021). ISSN: 0004-5411. DOI: [10.1145/3450745](https://doi.org/10.1145/3450745).